What Makes a Memory View Useful?

A Controlled Study of Deterministic Lifecycle Views for Long-Term Conversational Memory

Yuhang Jia
Department of Mathematical Sciences, Tsinghua University
jia-yh22@mails.tsinghua.edu.cn

September 2026

Abstract

Long-term conversational agents must decide not only what to store but what to show the model at query time. Recent work frames this as *query-conditioned forgetting*: track the lifecycle of each fact (which values superseded which) and suppress or expose the history depending on what the query asks for. We set out to study that mechanism on LongMemEval-S and report a negative result at its core, together with a positive one that displaced it.

We build a fully training-free pipeline: a frozen LLM extracts atomic (entity, attribute, value) triples per session, deterministic rules assemble them into per-attribute chains labelled with supersession / contradiction / co-occurrence relations, and a rule-based classifier routes each query to a current-state or full-history rendering of those chains. On the full 500-question benchmark under a single shared answer prompt and matched context budgets, **the query-conditioning mechanism contributes exactly nothing**: ablating it changes accuracy by 0.0000 (12 wins, 12 losses, McNemar $p = 1.00$), and on the 456 questions where the two renderings genuinely differ in text the accuracy difference is likewise exactly zero. The mechanism is inert here because the two renderings carry nearly identical answer content and because 425 of 500 questions carry no lexical tense cue at all.

What does work is the view as an object, plus three context-construction interventions that we derived by reading every residual error. Relevance-ordered view truncation and role-quota retrieval lift accuracy from 0.870 to 0.898; a deterministic quantity ledger, a temporal scaffold, and a narrow-and-deep rendering for advice-shaped questions lift it to **0.922**, against 0.838 for a no-forgetting full-memory baseline (+8.4 points, $b = 53$, $c = 11$, $p < 10^{-5}$), 0.828 for a budget-matched retrieval-only control (+9.4 points, $p < 10^{-5}$), and 0.894 for our reimplementation of a concurrent slot-supersession system (+2.8 points, $p = 0.038$). Every arm runs on all 500 questions with one fixed judge, and every intervention is budget-neutral to within 0.1%.

Two methodological findings generalize beyond this system. First, an uncontrolled difference in system-prompt wording between our baseline runner and our method runner manufactured 1.6 points of illusory gain, by making the baseline decline to answer on 51 of 470 answerable questions versus 19 for the method. Second, and more consequential for how such systems are developed: across 68 offline-validated claims about candidate improvements, 56 failed adversarial verification, and the dominant failure mode was that the offline proxy—gold-string coverage, evidence-set completeness, derivability—reproduced arithmetically while failing to predict answer correctness. Two interventions that offline analysis rejected turned out to be the single largest real gain we measured. We conclude that in this regime only paired online measurement licenses a design decision.

1 Introduction

An agent that talks to the same user for months accumulates a conversational history far larger than any context window. The standard response is a memory system: store the history in some structured form, retrieve a subset at query time, and answer from that subset. Most of the design effort in recent systems goes into *what to store and what to drop*—decay schedules, capacity eviction, consolidation, summarization.

A newer line of work argues the interesting decision is downstream of storage. Because a fact's value changes over time ("I moved to Berlin" after "I live in Munich"), the memory holds a *lifecycle*: a chain of values with supersession relations between them. Whether the old value should be shown depends on the query. Asked "where do I live?", the agent should suppress the superseded value; asked "where did I live before?", it must expose it. Two concurrent preprints make this separation explicit—one separating the retained archive from the query-conditioned view (Li and Li, 2026), the other modelling merge/update/contradiction relations across memory items (Yacoubi et al., 2026)—and both report substantial gains from lifecycle awareness.

We set out to study that mechanism in isolation: to make the query-conditioning decision explicit and auditable (a rule table rather than an LLM call), to measure the decision's own accuracy, and to quantify what misrouting costs. The study produced a clean negative result. Under a controlled comparison the mechanism contributes nothing measurable on this benchmark, and we can say precisely why. Pursuing the reason led to a different and effective design, and to a methodological lesson about how such designs should be validated.

Contributions.

1. **A controlled null result for query-conditioned lifecycle views** (§5.2). With storage, retrieval, budget, answer model, prompt, and judge all held fixed, removing query conditioning changes accuracy by 0.0000 on 500 questions. We localise the cause: the current-state and full-history renderings carry the gold answer in 186 and 188 of 500 questions respectively, overlapping in all but 10, so the routing decision can matter on at most 2% of the benchmark; and 85% of questions carry no lexical tense cue, defaulting to a single rendering regardless.

2. **A deterministic lifecycle view that does help, and an attribution control that isolates it** (§5.3). At an identical 20-turn retrieval budget, adding the view raises accuracy from 0.828 to 0.898 ($p < 10^{-5}$). The control differs from the method only in the presence of the view, so the gain cannot be attributed to a retrieval-configuration difference—a confound we found in our own earlier comparison.

3. **Three error-derived context interventions, each budget-neutral** (§4.3, §5.4): a quantity ledger that force-renders every quantity-bearing fact for aggregation questions, a temporal scaffold that treats dates as retrieval keys and pre-computes offsets, and a narrow-and-deep rendering for advice-shaped questions. Together they take accuracy from 0.898 to 0.922 and repair the one subtype where our method had been *worse* than the no-forgetting baseline.

4. **Two methodological findings** (§6). A system-prompt wording asymmetry between runners manufactured 1.6 points of illusory gain through differential abstention. And offline proxy metrics systematically failed to predict answer correctness: 56 of 68 offline-validated improvement claims did not survive adversarial recomputation, while two claims that offline analysis rejected produced the largest measured gain. We report the audit trail because we believe the failure mode is general to memory-system development, where full online evaluation is expensive and proxies are tempting.

We do not claim state of the art. Our judge and answer model differ from those used in published LongMemEval-S numbers, so our absolute scores are not comparable to them (§3.4); every comparison in this paper is internal, paired, and run under one fixed configuration.

2 Related Work

Memory management by storage policy. MemoryBank-style time decay (Zhong et al., 2024) and MemoryOS-style capacity eviction (Kang et al., 2025) decide what to keep; extraction-and-consolidation pipelines (Chhikara et al., 2025) and temporal knowledge graphs (Rasmussen et al., 2025) decide what shape to keep it in. We reimplement both as baselines. Under our configuration decay scores 0.798 and FIFO eviction 0.330 on 500 questions; the FIFO collapse is informative rather than a strawman result, since it shows how much of this benchmark depends on old sessions that a recency policy discards.

Lifecycle-aware memory. Two concurrent preprints (Li and Li, 2026; Yacoubi et al., 2026)—both arXiv v1, September 2026, no venue at the time of writing; we treat them as concurrent work—are the direct antecedents of our study. The first separates a retained source archive from a query-conditioned view, links same-slot replacements so superseded values are suppressed in current-state contexts, and lets an intent signal make early evidence available again. The second models merge / update / contradiction / no-action relations between atomic memories with provenance, and rebuilds relation-aware evidence units at retrieval time, exposing current, historical, supporting and conflicting evidence to the reader.

Our positioning relative to them is narrower than we originally planned, and we state the retraction explicitly. The supersession-versus-contradiction distinction is not ours to claim: the second system already labels four relation types via LLM judgement. The idea of re-admitting old values under a historical intent is not ours either: the first system already has an intent switch. What remains ours is (i) rule-based, auditable relation labelling and query routing, where both antecedents use LLM calls, (ii) an explicit multi-valued-attribute vocabulary that neither has, and (iii) the measurement contribution—evaluating the routing decision itself, and reporting that on this benchmark it does not matter.

Reimplementation as a baseline. We reimplement the first system's design as our `rdforget` arm (§3.3): slot chains where only the active value is rendered and superseded values are excluded by default, a binary historical-intent switch, lexical fallback retrieval, and a single fixed view budget. It is a strong baseline, scoring 0.894 against the 0.838 full-memory baseline ($p = 0.0002$), and our method beats it by 2.8 points ($p = 0.038$). Reimplementing it also surfaced a protocol weakness we report rather than exploit quietly: its intent function combines lexical cues with the benchmark's gold task-type label. In our reimplementation that label decides 189 of the 223 questions routed to historical—85% of the routing decisions come from metadata unavailable in deployment.

Related conditioning axes. Task-conditioned memory views (Chen et al., 2026) and systems that map a question to a temporal intent before executing deterministically (Dai et al., 2026) both predate our query-shape conditioning; we cite them as precedent for the conditioning idea rather than claiming it. Ontology-driven lifecycle policies (Mullick and Tüzün, 2026), write-time revocation of stale precedents (Zhou et al., 2026), and benchmarks that score current-versus-superseded state tracking (Fan et al., 2026) are adjacent developments from the same period.

3 Experimental Setup

3.1 Benchmark

LongMemEval-S (Wu et al., 2025), **cleaned** variant (the original is deprecated; noisy sessions there affect answer correctness). We pin the file by hash, following the one prior study we know of that does (Shen, 2026); results on the cleaned and original releases are not directly comparable. 500 questions over six capability types: single-session-user (70), single-session-assistant (56), single-session-preference (30), multi-session (133), knowledge-update (78), temporal-reasoning (133). Thirty questions (`_abs` suffix) have unanswerable golds and are reported separately, since answering them correctly means declining. Haystacks total 23,867 session instances, of which 18,464 are unique by content—we exploit this for caching (§4.1). Each question carries a timestamp, and every session is dated to the minute, which makes the relation rules in §4.2 a total order rather than a partial one.

We report every arm on all 500 questions. No subsampling: a 60-question stratified sample was used during development only, and we treat it as methodologically contaminated because it drove design decisions.

3.2 Models and judging

Answer model: `deepseek-flash`, 6144 max tokens, reasoning enabled, identical across all arms. Extraction: `deepseek-flash` with reasoning disabled (§4.1). Judge: `deepseek-v4-pro` with one fixed prompt, temperature 0, single vote. Judge and answerer are deliberately different models, so no arm is graded by its own family.

Judge determinism was checked: across the seven arms of §3.3 there are 604 question instances where two arms produced normalized-identical predictions, and the judge returned the same verdict on every one of them; pooling every run we made, including the superseded ones of §6.1, gives 1,696 such instances and still zero disagreements. This bounds judge stochastic noise near zero. It says nothing about judge *validity*, which we discuss in §3.4.

3.3 Arms

All arms share one answer model, one judge and matched context budgets, and differ only in what context they build. One caveat belongs here rather than in a footnote: of the ten rows in §5, seven were produced after the prompt unification of §6.1, while `decay`, `fifo` and `view_only` were not re-run and retain the earlier baseline wording. We mark them in the main table. The effect of that wording is to increase declining, which costs accuracy, so those three rows are if anything pessimistic; every comparison this paper draws a conclusion from—the null ablation, the attribution control, `full`, `rdforget`—is between arms that share one prompt string.

Our method appears in three versions: **v4** (view + BM25 top-20), **v5** (v4 + relevance-ordered view truncation + role-quota retrieval), **v6** (v5 + the three interventions of §4.3).

The `no_lifecycle` control exists because our first comparison was confounded: the method used a 20-turn BM25 budget plus a view while `full` used 30 turns, so a method-versus-`full` gap mixes a view effect with a retrieval-budget effect. The control removes that confound by running the method's own retrieval path and deleting only the view. The run configuration is not stamped into the stored rows, which is a reproducibility gap we note in §7; we identify it from the reconstructed context sizes and the per-call prompt lengths in the ledgers.

Arm	Context construction
`full`	No forgetting. BM25 top-30 turns over all sessions.
`decay`	MemoryBank-style: sessions kept by exponential recency (30-day half-life, 0.05 threshold), then BM25 top-30.
`fifo`	MemoryOS-style: most recent 15 sessions only, then BM25 top-30.
`rdforget`	Reimplementation of the concurrent slot-supersession system (§2): active-value-only slot view, superseded values excluded unless a binary intent switch fires, plus BM25 top-20 turns.
`no_lifecycle`	**Attribution control.** The method's own v5 retrieval (role quota plus session augmentation, up to 26 turns), with the view removed. Identical retrieval configuration to our method, minus the view.
`no_qc`	**Core ablation.** Our method with the view always rendered in full-history mode, i.e. no query conditioning.
`view_only`	Our method's view with no raw turns, measuring view self-sufficiency.

Table 1: The seven arms. Everything outside this column—answer model, decoding parameters, retry policy, judge—is held fixed.

3.4 What our numbers can and cannot be compared to

Published LongMemEval-S results use GPT-4o-mini with the official per-type judge prompts; we have access to a DeepSeek endpoint only. Our absolute scores are therefore **not** comparable to published numbers, and we make no such comparison. Third-party reproductions of the same systems on this benchmark differ by more than 20 percentage points (Wang, 2026), so cross-paper absolute numbers are of limited value regardless. We also report token-F1 alongside accuracy for protocol compliance, but we argue against reading it as corroboration. It does not track the accuracy ranking: `decay` scores 0.270 against `full`'s 0.226 and `no_lifecycle`'s 0.229 while sitting three to four accuracy points below both. It is also length-confounded, and answer length varies by 45% across arms (mean 21.2 scored tokens for `fifo` and 22.3 for `decay` against 30.7 for `full` and 28.6 for ours), so the metric mixes verbosity into every pairwise comparison. We report it; we do not interpret it.

We use the benchmark's gold metadata—`question_type`, `answer_session_ids`, per-turn `has_answer` flags—for diagnosis only, never inside any method. The one arm that reads a gold label at inference time is the `rdforget` reimplementation, faithfully, because the system being reimplemented does so.

4 Method

4.1 Extraction and caching

For each session, a frozen LLM extracts atomic facts as `entity | attribute | value` lines, with an instruction emphasizing facts that *update* earlier state, including updates mentioned in passing or about third parties. Output is parsed into triples; an unparseable response is retried once with a stricter format instruction, then once with reasoning enabled, and a session that still fails is dropped with an audit record rather than silently cached as empty.

Two implementation choices matter for cost and for reproducibility.

Content-hash session caching. Extraction is cached keyed on the md5 of the session text, not on the question id. Because 23,867 session instances reduce to 18,464 unique contents, this removes

23% of extraction work. It also decouples relation labelling from extraction: dates are stamped onto cached triples at assembly time, so a change to the relation rules costs no API calls. The extraction prompt deliberately contains no date, which keeps the cache exactly prompt-equivalent.

Reasoning disabled for extraction. Both available models are reasoning models, and reasoning tokens bill as completion. With reasoning enabled, 25.5% of extraction calls (740 of 2,900 ledgered calls) hit the completion cap; 662 of those returned no content at all and the other 78 returned a truncated fragment, and either way the failure was being cached silently. Disabling reasoning cut output per call by 24× (mean 2,740 to 117 completion tokens, 27× on medians) and removed the cap failures.

The fact-level trade is worth stating plainly, because it is not free. Comparing the two extraction generations on the 2,264 sessions where the reasoning-enabled pass did *not* cap out, reasoning-off extracts 18.6% fewer facts (22,752 against 27,937). It is net-positive only because it eliminates the cap failures: across all 2,793 sessions present in both generations the ratio is 0.886, and the empty-extraction rate falls from 18.9% to 6.8%. We took the trade because a silently empty session is worse than a thinner one—it removes an entire session from every view built from it—but a system with budget for reasoning-enabled extraction and a large enough completion cap should expect a denser store than ours.

4.2 Chain assembly and relation rules

Facts sharing a normalized (entity, attribute) key form a chain, sorted by (date, session index). Relations are assigned by deterministic rule—no LLM call, and the rules are printable in full, which is the auditability property we wanted:

- **co-occurring**: two different values from the *same* session. Usually paraphrase or elaboration, not conflict.
- **contradiction**: different values from *different* sessions carrying the *identical* timestamp, hence unorderable.
- **supersession**: a different value at a strictly later timestamp.
- **accumulative**: the whole chain, when the attribute is in a multi-valued vocabulary (hobbies, goals, concerns, ...) or when one session lists three or more distinct values. Such chains coexist rather than supersede.

An earlier version of these rules was wrong in a way worth recording, because it inverted the paper's own taxonomy. It compared (date, session index) tuples after sorting by the same key, so `contradiction` could only fire on exact ties, i.e. within a single session. That makes the defect structural rather than statistical: under the old rule *every* contradiction label is same-session by construction, and genuine same-day cross-session conflicts are all labelled supersession. Replaying the old rule over the shipped extraction cache produces 11,608 contradiction labels across the 500 questions, all 11,608 of them same-session. Every `[CONFLICT]` marker shown to the answer model was therefore a false positive. Since timestamps include the minute, the corrected rule above is a total order.

The view renders one line per chain. In **current** mode, each chain shows the values of its latest session, with a `[CONFLICT]` marker only for genuine unorderable conflicts; in **historical** mode, the full evolution $v_1 \to v_2 \to v_3$ with dates. Figure 1 shows both renderings of three real chains, rebuilt offline from the extraction cache.

```
(a) genuine supersession                                        [qid 945e3d21]
    relations  : initial, supersession
    current    : [2023/08/25 (Fri) 19:36] user | is planning a trip to = Hawaii
    historical : user | is planning a trip to:
                   San Antonio (2023/08/02) -> Hawaii (2023/08/25)

(b) accumulative, fired by the >=3-values-in-one-session rule [qid 6aeb4375]
    relations  : accumulative x4
    current    : user | recently got = a new pair of black boots from Zara
                    (2023/07/16); a little white dress from Madewell (2023/07/16);
                    a soft cashmere scarf from a spa boutique (2023/07/16);
                    hydrangea cuttings from their neighbor's garden (2023/09/28)
    historical : identical -- accumulative chains never collapse

(c) spurious chain: a generic verb attribute merges referents  [qid d7c942c3]
    relations  : initial, supersession, cooccurring, supersession, supersession
    current    : [2023/05/14 (Sun) 21:13] user | wants = a nice pair of shoes
                   to match a new outfit
    historical : user | wants: mom to use the grocery list app (2023/03/11)
                   -> ordered list of lead guitar solos to learn / daily 1 hour
                      guitar practice routine for soloing and shredding (2023/03/22)
                   -> certbot command to run once a day via CRON (2023/05/07)
                   -> a nice pair of shoes to match a new outfit (2023/05/14)
```

Figure 1: Three real view lines, rebuilt offline from the session cache; text is verbatim, only the indentation is ours. Case (a) is the behaviour the design targets: current mode suppresses the superseded value. Case (b) shows why the accumulative rule is necessary—without it, four things the user acquired would be read as three supersessions and only the last would survive current mode. Case (c) is the failure mode the quantity ledger of §4.3 exists to contain: `wants` is a generic verb attribute, so unrelated referents share one chain, and current mode prints only the most recent, deleting the rest. Case (c) is also part of why the null result of §5.2 holds: much of what the two modes disagree about is chaining noise of this kind rather than genuine state evolution.

4.3 Context construction

The answer context has up to four sections: the fact view, a quantity ledger, retrieved raw turns, and a temporal scaffold. Every component below is budget-neutral: added characters are deducted from another section, and we verify total context length per question offline before running.

Relevance-ordered view truncation (v5). The rendered view averages 30,644 characters against a 20,000-character cap—the cap binds on 500 of 500 questions—and the earlier implementation truncated in alphabetical order of `entity::attribute`, discarding 34% of chains on a criterion unrelated to the question. We score chains against the question by BM25 and let relevance compete for the cap, then restore the original stable order so that relevance does not leak as a positional cue.

Role-quota, session-augmented retrieval (v5). Instead of a flat BM25 top-20, we take the top 14 user turns and top 6 assistant turns, then add the user turns of the highest-scoring session under a session-level BM25 over user text. Gold-evidence-turn recall rises substantially on the types that need it (preference $0.500 \rightarrow 0.727$, multi-session $0.644 \rightarrow 0.728$, temporal $0.783 \rightarrow 0.838$).

Quantity ledger (v6, fires on 175/500). Aggregation questions failed not because the model cannot add, but because operands were absent from the context, by two mechanisms we traced in the code. *Relevance pruning*: the view saturates its cap on all such questions, and a decisive quantity lands on the discarded side. *Current-mode collapse*: a generic verb attribute merges unrelated quantities into one chain labelled supersession, and current mode prints only the latest group, so the view itself destroys earlier operands. One example: the chain `user::has completed` holds "8 edX courses (05/21)" and "12 courses on Coursera (05/27)"; current mode rendered only the latter, and the model answered 12 where the gold was 20. The ledger force-renders every quantity-bearing fact in the store, de-duplicated, date-ordered, annotated with days relative to the question date, and read from the full chain history so collapse cannot hide an operand. Its characters are deducted from the view budget; measured total context change is -0.002%.

Temporal scaffold (v6, fires on 179/500). Of the residual temporal errors, most had their gold evidence turn *absent* from the context, and the absence had a shape: questions specify offsets ("two weeks ago") or windows ("during the past month"), which share no lexical content with the evidence turn, so BM25 can never retrieve it. Worse, within a same-day session family the event time exists only inside the text ("3 weeks ago"), while every line prefix shows the same date—the ordering information is simply not present in the context. The scaffold treats dates as retrieval keys (anchor day ± 3 days, in-window event reports), renders a de-duplicated compact timeline with in-text event dates preferred and signed day offsets relative to the question date, and pre-computes the implied interval when both endpoints are identifiable. We use the budget-neutral variant, which trims the lowest-scoring retrieved turns by the scaffold's character count.

Narrow-and-deep rendering for advice-shaped questions (v6, fires on 29/500). Preference questions were the one type where our method scored *below* the no-forgetting baseline. The cause was not retrieval: in five of six cases where the baseline was right and we were wrong, the gold evidence turn was already in our context. Preference golds are rubrics—a correct answer must build on the user's specific on-record details rather than generalities—and our context was wide and shallow: the view saturated the cap on all 30 questions with 94% of its lines sharing no content word with the question, occupying half the context, while 26 retrieved turns scattered across a dozen sessions. The model then assembled an answer from an irrelevant attribute list. For advice-shaped questions, detected by a surface speech-act rule (29/30 preference questions, zero false positives on the other 470), we tighten the view cap to 2,500 characters and spend the freed budget on the session neighbourhoods of already-retrieved turns, with that question's v5 context length as a hard ceiling.

Gate verification. Each intervention is gated, and we verify offline that with its flag off the constructed context is byte-identical to v5, and that budget parity holds: quantity ledger $1.0000\times$, advice rendering $0.9999\times$, temporal scaffold $0.9994\times$, all three combined $0.9993\times$, with zero construction errors on 500 questions. Byte-identity outside the gate also means the arm only needs to be *run* on gated questions, which is what made these comparisons affordable: the combined arm required 286 online questions rather than 500, and the paired test is unaffected because the out-of-gate predictions are identical by construction.

4.4 Query classification

The tense classifier is a printable rule table over the question text: historical patterns (previously, used to, originally, before, earlier, "previous X" excluding conversational meta-references like "previ-

ous chat"), then current patterns (currently, now, latest, still), else neutral, which is routed to the current-state view. Over 500 questions it labels 44 historical, 31 current, and 425 neutral. That distribution is the first indication of the null result in §5.2: the mechanism makes a non-default decision on 15% of the benchmark.

5 Results

All arms, 500 questions, one shared prompt, one fixed judge. Significance is exact McNemar on discordant pairs; b counts questions our method wins and the reference loses, c the reverse.

5.1 Main table

Arm	Acc (500)	Acc (470 non-abs)	Abstention (30)	Token-F1
`fifo` (MemoryOS-style)†	0.330	0.287	1.000	0.163
`view_only`†	0.500	0.470	0.967	0.165
`decay` (MemoryBank-style)†	0.798	0.789	0.933	0.270
`no_lifecycle` (attribution control)	0.828	0.821	0.933	0.229
`full` (no forgetting)	0.838	0.834	0.900	0.226
Ours v4	0.870	0.870	0.867	0.258
`rdforget` (concurrent-system reimpl.)	0.894	0.898	0.833	0.266
`no_qc` (no query conditioning)	0.898	0.900	0.867	0.269
Ours v5	0.898	0.896	0.933	0.269
Ours v6	**0.922**	**0.926**	0.867	0.273

Table 2: Accuracy by arm on all 500 questions, on the 470 answerable questions, on the 30 abstention questions, and token-F1. One answer model and one judge across every row. † marks the three arms that were not re-run after the prompt unification of §6.1 and therefore carry the earlier, more decline-prone baseline wording; the remaining seven share one prompt string.

Comparison	Δ	b	c	McNemar p
v6 vs `full`	+0.084	53	11	$< 10^{-5}$
v6 vs `no_lifecycle`	+0.094	58	11	$< 10^{-5}$
v6 vs `decay`†	+0.124	70	8	$< 10^{-5}$
v6 vs `rdforget`	+0.028	27	13	0.038
v6 vs `no_qc`	+0.024	23	11	0.058
v6 vs v5	+0.024	23	11	0.058
v5 vs `full`	+0.060	44	14	0.0001
v5 vs `no_lifecycle`	+0.070	45	10	$< 10^{-5}$
v5 vs `no_qc` *(core ablation)*	**+0.000**	**12**	**12**	**1.000**
`rdforget` vs `full`	+0.056	41	13	0.0002

Table 3: Paired comparisons, exact McNemar on 500 questions. The boldface row is the core ablation: removing query conditioning changes nothing. † the `decay` row compares across the prompt change of §6.1 and is not a clean single-variable comparison; we report it for completeness and rest no claim on it.

Paired by type, v6 versus `full`: multi-session $b = 28$, $c = 5$, $p = 0.0001$; temporal-reasoning $b = 17$, $c = 3$, $p = 0.0026$; the other four types are at or near ceiling and move by at most a few

Type	n	full	rdforget	no_lifecycle	no_qc	v5	**v6**
single-session-user	70	0.986	1.000	0.971	0.986	0.986	0.986
single-session-assistant	56	0.982	0.982	0.964	0.982	0.982	0.982
single-session-preference	30	0.667	0.600	0.533	0.700	0.600	**0.733**
multi-session	133	0.684	0.812	0.684	0.827	0.820	**0.857**
knowledge-update	78	0.936	0.949	0.962	0.936	0.987	0.974
temporal-reasoning	133	0.835	0.917	0.827	0.910	0.910	**0.940**

Table 4: Accuracy by question type. Gains concentrate in multi-session and temporal-reasoning; the two single-session types are at ceiling for every arm that retains old sessions.

questions (knowledge-update $b = 4$, $c = 1$, $p = 0.375$; preference $b = 4$, $c = 2$, $p = 0.688$; the two single-session types are identical).

5.2 The core mechanism contributes nothing

Removing query conditioning entirely—rendering the view in full-history mode for every question—changes accuracy by **0.0000** (12 wins, 12 losses, $p = 1.00$). This is not a small effect measured imprecisely; it is zero, and the structure underneath it is equally clean.

The classifier routes 456 of 500 questions differently from the always-historical ablation, and those 456 questions genuinely receive different view text: rebuilding both renderings offline from the complete session cache shows a median of 20 differing lines out of 209 on those questions, a 9.6% perturbation of the view (over all 500 the medians are 19 of 208, 9.1%). On exactly those 456 questions the two arms score **identically**, with 11 flips in each direction. The only remaining discordance is one question each way among the 44 whose contexts are byte-identical between the arms—which is to say, answer-model nondeterminism on inputs where nothing was ablated. The discordance is balanced, $b = c$, on each of these three subsets. It is not balanced within every question type: routing hands the ablation four knowledge-update questions and takes back three preference and two abstention ones, which is the shape you would expect from noise at $n = 78$ and $n = 30$, and which we report rather than aggregate away.

Three facts explain why the mechanism is inert on this benchmark:

1. **The two renderings carry the same answers, as far as a string test can tell.** The gold answer string appears in the current-state view for 186 of 500 questions and in the full-history view for 188, and only 4 are current-only while 6 are historical-only—10 questions, 2% of the benchmark, on which a verbatim-gold test can even detect a routing effect. This is a detectability bound, not a causal ceiling, and we flag one case that escapes it: the aggregation example of §4.3 (`67e0d0f2`) is scored as gold-in-both because the gold "20" also occurs inside dates, yet current mode answers 12 and the always-historical ablation answers 20 correctly. Questions whose gold is derived rather than quoted are outside the test's reach; the measured null is not affected, but the 2% figure should not be read as a bound on what routing could ever do.

2. **Almost nothing is routed.** 425 of 500 questions carry no lexical tense cue and fall to the default. The mechanism makes a genuinely historical decision on 44 questions, of which only 13 are knowledge-update and 13 temporal-reasoning—the two types the design targets.

3. **Raw turns rescue misrouting.** On the five questions that genuinely needed the historical rendering but were routed to current, our method still answered all five correctly, because the

20 retrieved raw turns contained the evidence regardless.

We also note the ablation tests the mechanism in the less informative direction. Always-historical differs from our routing on 456 questions; the complementary always-current baseline would differ on only 44, which bounds any effect it could show at 8.8% of the benchmark. A sharper design would compare against always-current and would repeat each arm to establish the nondeterminism floor, which we measure at roughly 2/44 flips on byte-identical inputs.

The honest reading is that the tense axis is the wrong conditioning variable for this benchmark—not that conditioning is a bad idea in general. Its surface area is too small, and where it does apply the two renderings agree.

5.3 The view itself is load-bearing

Against the budget-matched control that differs only by the absence of the view, v5 gains 7.0 points (0.898 vs 0.828, $b = 45$, $c = 10$, $p < 10^{-5}$) and v6 gains 9.4 points. This is the comparison we consider the paper's positive result, because the control holds retrieval configuration, budget, prompt, model and judge fixed.

The view is not self-sufficient: `view_only` scores 0.500, far below both. The view and the retrieved turns are complementary—measured over 470 answerable questions, the gold string appears in the view but not the turns for 39 questions, and in the turns but not the view for 55. A system that keeps only one of the two loses either way.

5.4 Interventions

Each intervention was A/B'd on its gated subset against v5, with out-of-gate predictions identical by construction:

Intervention	**Gated n**	**v5 → v6 (gated)**	b	c	p	**Projected on 500**
Quantity ledger	175	0.903 → 0.943	7	0	0.016	0.898 → 0.912
Temporal scaffold	179	0.888 → 0.922	9	3	0.146	0.898 → 0.910
Advice rendering	29	0.586 → 0.690	7	4	0.549	0.898 → 0.904
Combined	286	0.864 → 0.906	23	11	0.058	0.898 → **0.922**

Table 5: Gated A/B of each context intervention against v5. Out-of-gate predictions are byte-identical by construction, so the paired test on the gate is the paired test on 500.

The quantity ledger is the cleanest: 7 questions repaired, **0 broken**, at a measured context change of −0.002%. The individual temporal and advice interventions are directionally positive but not individually significant at their gated sample sizes; the combined arm is what we report as the method, and its subtype effects are where the design intended them (multi-session 0.820 → 0.857, temporal 0.910 → 0.940, preference 0.600 → 0.733).

The three gates are nearly disjoint (ledger ∩ advice = 0, advice ∩ temporal = 0, ledger ∩ temporal = 97 questions but only 1 overlapping repair), and no question repaired by one intervention is broken by another, which is why they stack.

Stacking is nonetheless not additive, and we report the shortfall rather than only the sum. The 97 questions in both the ledger and the scaffold gate receive both treatments only in the composed arm, and composition costs more than the parts: the individual arms break 7 questions between them while the composed arm breaks 11, with 8 questions broken *only* under composition; individual repairs total 22 against the composed arm's 23, so 2 questions repaired in isolation are lost when the

interventions combine. The composed arm also gives back one knowledge-update question relative to v5 (0.987 → 0.974), the one subtype where v6 is behind its own predecessor. The headline 0.922 already includes all of this.

5.5 Where the errors actually are

We hand-classified every error in the v4 arm into the three categories our study was designed around, plus a catch-all. This classification was done on the pre-prompt-fix v4 and `full` runs of §6.1 (67 and 91 errors respectively), before the prompt was unified; the corresponding unified-prompt arms have 65 and 81 errors, and we have not re-labelled them by hand. The composition of the error set is what the section argues about, and that is stable across the two runs. The result is uncomfortable and we report it prominently: **the three planned categories account for** 9.0% **of errors** (stale-fact 3, history-loss 0, conflict-confusion 3 out of 67). The bulk is retrieval-miss (35) and aggregation (11), with abstention behaviour and grading boundaries making up most of the rest. The accuracy gap against the baseline is carried by retrieval and aggregation, not by the temporal error classes the lifecycle story predicts. The one clean directional win in the planned categories is history-loss, 2 errors for the baseline against 0 for us.

This diagnosis is what produced §4.3, and it is the reason the paper's analytical axis is retrieval completeness and aggregation rather than temporal error typology.

After v6, 39 errors remain (35 answerable): multi-session 17, preference 8, temporal 6, and 4 across the three near-ceiling types.

5.6 Abstention

On the 30 unanswerable questions our final method scores 0.867 (26/30), below `full` at 0.900 and `decay`, `no_lifecycle` and our own v5 at 0.933. Our four failures are `09ba9854_abs`, `a96c20ee_abs`, `gpt4_70e84552_abs`, `gpt4_93159ced_abs`; of these, the first two also defeat `full`, so exactly two questions separate us from the baseline on this subset. At $n = 30$ that is a trend, not a result, and no pairwise comparison here approaches significance.

Two observations keep us from either dismissing or overstating it. Against us: the direction is consistent across our method versions, and it is the expected failure mode of injecting a synthesized view—the view supplies plausible-looking material, so the model answers where it should decline. In our favour: measured as a decline *rate*, our arm is the most decisive of all. Counting declines with a fixed lexical detector (`v5_guards.is_decline`, which agrees with the judge on 28 of the 30 abstention questions), v6 declines on 26 of 30 unanswerable questions and on only **11 of 470 answerable ones**, against 28/30 and 31/470 for `full` and 28/30 and 66/470 for `decay`. Since a decline on an answerable question is never scored correct, the subset gap is a precision/recall trade: `decay` buys its two extra abstention questions with 55 more declined answerable ones. Part of `decay`'s declining is the prompt artefact of §6.1 rather than a property of time decay, since that arm was not re-run; the comparison against `full`, which was, is the one to weight.

We had attributed the v5-to-v6 abstention loss (0.933 → 0.867) to the quantity ledger, on the reasoning that force-rendering quantities gives an unanswerable question something to be answered *from*. Our own gated arms refute that. In the ledger-only arm the abstention questions inside its gate go 13/14 → 13/14, and the scaffold-only arm is flat at 11/11; both of the two lost questions (`gpt4_70e84552_abs`, `gpt4_93159ced_abs`) are answered correctly by the single-intervention arms and fail only in the composed one. The loss is a composition effect that no single intervention reproduces, and we leave it unexplained rather than assign it to a component that the A/B clears.

6 Two Methodological Findings

6.1 A prompt-wording asymmetry manufactured 1.6 points

Our first main table reported +4.8 points for the method over the no-forgetting baseline ($p = 0.006$) and +36.7 points on the preference subtype. Both were substantially artifacts.

We had unified the answer model, token budget, and empty-answer retry across arms, but not the system prompt *text*: the baseline runner's wording ("if the memory contains no relevant information, say you don't know") induced far more declining than the method runner's. On 470 answerable questions the baseline declined 51 times under that wording against 19 for the method, and 44 of those 51 declines were scored incorrect (the other 7 happened to contain enough of the gold to pass). Twenty-nine of the method's 47 wins over the baseline were questions where the baseline declined. After unifying the prompt across all arms, the baseline's declines on answerable questions fell from 51 to 31 and its accuracy rose from 0.818 to 0.838, while the method's advantage fell from +4.8 points ($p = 0.006$) to +3.2 points ($p = 0.052$) at the same method version.

The preference subtype is the sharpest case, and it does not merely shrink—it reverses. Before the fix the method led the baseline by +36.7 points on that subtype (0.700 vs 0.333); after it, the method is *behind* by 3.3 points (0.633 vs 0.667). Nearly the entire apparent gain had been the baseline declining to give advice. That reversal is what §4.3's advice rendering was built to repair, and §5.4 reports the repair ($0.600 \rightarrow 0.733$ at v6).

The comparison that was *not* affected is the attribution control, which had shared the method's prompt all along. This is the reason we consider §5.3 the robust positive result and treat method-versus-`full` as secondary.

We draw a narrow lesson: matching the answer model, decoding parameters, and retry policy across arms is not sufficient. Prompt wording modulates abstention, abstention is scored as error, and any arm whose prompt differs is not a controlled comparison. Every number in this paper comes from arms sharing one prompt string.

6.2 Offline proxies did not predict answer correctness

We validated candidate improvements offline before paying for online runs, using gold-string coverage, gold-evidence-set completeness, and per-family derivability—proxies chosen because they are cheap and gold-grounded. We then ran an adversarial verification pass in which independent agents attempted to refute each claim by recomputing it. **Of 68 claims, 56 were refuted**, and the dominant refutation was not arithmetic error—the numbers almost always reproduced exactly—but that *the proxy did not track answer correctness*: coverage metrics were gamed by the intervention that optimized them, completeness gains coexisted with accuracy-neutral or negative context changes, and derivability was measured on questions whose gold is a derived number that appears nowhere verbatim.

The decisive evidence is that the adversarial pass rejected relevance-ordered view truncation and role-quota retrieval, on the grounds that their offline gains vanished on the context the model actually receives. Run online, those two changes are the largest single step we measured, $0.870 \rightarrow 0.898$, and they are what lifts the method to +6.0 points over the no-forgetting baseline ($p = 0.0001$ for v5 vs `full`). We state the two tests separately because they are separate: the paired v4-to-v5 step is $b = 32$, $c = 18$, $p = 0.065$, which is no more individually significant than the v6 step we flag in §7. The point here is not that the step clears a threshold—it is that offline analysis ruled it out, in the direction opposite to every online measurement of it. Offline analysis was confidently wrong in both directions—it rejected what worked, and it endorsed variants that never earned an online test.

We do not conclude that offline analysis is useless. Reading failing cases was indispensable: every intervention in §4.3 came from reading errors verbatim and naming a mechanism in the code, and two real implementation defects (alphabetical view truncation, the inverted contradiction rule) were found that way. What failed was using a proxy *metric* as a decision procedure. Our practice by the end was: read the errors to design, verify offline only that the code produces the intended context change and preserves budget, and let a paired online A/B decide. Gating made that affordable—a gated intervention only needs online runs on the questions it touches.

We also note the cost asymmetry. The adversarial verification pass consumed roughly 11 million tokens of analysis and produced a wrong conclusion; the online A/B that overturned it cost about 12 CNY. In a regime where a full evaluation arm is this cheap relative to deliberation, measurement should come first.

What we can and cannot hand over here. This subsection is the one part of the paper whose evidence we cannot ship. The verification pass survives as roughly 120 analysis scripts under `audit/`, `verify/` and `vfy2/`, but we kept no claim register mapping each claim to its verdict, and the token and cost figures above are recollections of the development process rather than quantities recoverable from the artifacts. The two concrete cases are fully documented—the alphabetical view truncation and the inverted contradiction rule, both found by reading errors, both fixed in the shipped code—and the decisive comparison is reproducible: v5 against v4, online, on all 500 questions. The 68/56 tally should be read as our report of our own process, not as a measurement a reader can check, and we would not ask anyone to weight it as evidence.

7 Limitations

Single benchmark, single backend. Everything here is LongMemEval-S cleaned with one answer model and one judge. The null result of §5.2 is a statement about this benchmark's composition—85% of its questions carry no tense cue, and its two view renderings agree on almost all answers—not a general claim about query-conditioned memory. A benchmark built to stress tense distinctions could well show the mechanism working.

Judge protocol. One judge model, one generic prompt, single vote. We store full predictions for all arms, so multi-vote re-judging, per-type official prompts, and second-judge calibration are cheap post-hoc recomputations; we have not run them. Our absolute numbers are not comparable to published ones (§3.4).

Underpowered subtypes. Knowledge-update, the type the lifecycle story most directly targets, cannot be tested at this sample size: the baseline already answers 68 of 72 non-abstention KU questions correctly, and our method and the baseline disagree on only 5 questions total, where the smallest attainable exact two-sided p is 0.0625. No possible outcome of that comparison could have been significant. The same holds for the two single-session types, which are at ceiling. Preference ($n = 30$) and abstention ($n = 30$) are too small for the effects we observe there to be established.

The combined arm is not individually significant. v6 versus v5 is $p = 0.058$. We report it as the method because its component that is significant (the quantity ledger, $p = 0.016$, zero regressions) is the largest contributor and because the three gates stack without conflict, but a confirmation run with repeats would strengthen it.

Run configuration is not stamped into the rows. A stored row records the question, the verdict and the prediction, but not the environment flags the arm ran under. For two arms—the attribution control and the core ablation—we recover the configuration from reconstructed context sizes and ledger prompt lengths rather than from a recorded field. A row format that stamped the flag set would have made this a lookup instead of an inference, and we recommend it.

One arm is not fully self-identifying. The two temporal-scaffold variants we implemented—one at budget ratio 1.0151, the budget-neutral one at 0.9994—fire on the identical 179 questions, and the stored per-question rows do not record which variant produced them. We report the budget-neutral variant, which is what the run configuration specifies, but the artifacts alone cannot distinguish the two. Every other arm is identifiable from its rows.

No repeats. Every arm was run once. We measure the nondeterminism floor indirectly at roughly 2 flips per 44 byte-identical inputs, but arm-level variance is unquantified.

Extraction is a pipeline dependency. Facts the extractor misses cannot appear in the view, and the pipeline cannot tell us how often that happened. Of the 18,464 cached sessions, 1,257 (6.8%) hold an empty fact list; our code caches a legitimate "no facts here" and a three-attempt give-up identically, so those two are indistinguishable after the fact, and the give-up audit log the code appends to is not in our artifact set. Paraphrased or in-passing updates are the known weak spot, and extraction-miss appears in our error typology.

8 Conclusion

We set out to study query-conditioned lifecycle views and found that on LongMemEval-S the query-conditioning component contributes exactly nothing, for reasons we can localise in the benchmark's composition and in the view's content. What carries the gain is the view as an object—a compact, deterministically constructed, relevance-ordered rendering of per-attribute value chains—together with three interventions derived by reading residual errors: a quantity ledger for aggregation, a temporal scaffold that treats dates as retrieval keys, and a narrow-and-deep rendering for advice-shaped questions. At matched context budget the result is 0.922 against 0.838 for a no-forgetting baseline and 0.828 for a retrieval-only control differing only by the view.

The two negative findings are, we think, the more transferable contributions. A difference in system-prompt wording between arms manufactured a point and a half of illusory gain through differential abstention, which suggests that controlled comparison in this area needs to fix the prompt string, not just the model. And offline proxy metrics systematically failed to predict answer correctness, to the point of rejecting the largest real improvement we found. Memory systems are evaluated expensively and designed cheaply, which creates a strong pull toward proxy-driven development. Our experience is that the pull should be resisted: gate each intervention, verify offline only that the context changes as intended and the budget holds, and let a paired online comparison decide.

A Reproduction

Pipeline: `common.py`, `lifecycle.py`, `query_classifier.py`, `run_qc.py`, `run_baseline.py` and `run_rdforget.py`, all under `code/`; interventions in `code/{v6_agg,v6_tr}.py` and the gated branches of `run_qc.py`, each behind an environment flag (`QC_V5`, `QC_V6_AGG`, `QC_V6_SSP`,

QC_V6_TR). Runners: run_full500.py <arm> <workers> <tag> for full arms, run_v6_subset.py <FLAG> <workers> for gated subsets. Gate and budget verification: v6_gates.py (rebuilds every arm's context offline with the API client stubbed to raise, so a cache miss surfaces as an error rather than a charge). Coverage diagnostics: coverage_diag.py. Per-question rows for every arm: results/full500_*.jsonl and results/v6_*.jsonl. Every row carries qid, question type, judge verdict, token-F1 and the full prediction; the query label and view mode are carried by the method arms and the v6 files, the rdforget rows carry its binary intent flag instead, and the full, decay and fifo rows carry neither, since those arms build no view. The v6 arm is a composition and is not stored as one file: it is full500_qc_v5.jsonl on the 214 ungated questions and v6_COMBINED.jsonl on the 286 gated ones, the latter set being exactly the union of the three gates recorded in results/v6_gates.json.

Dataset: LongMemEval-S cleaned, sha256 d6f21ea9...c3a442. Extraction cache: 18,464 content-hashed session extractions under results/lifecycle_cache/sessions_nt/, sufficient to rebuild every view offline with no API calls. API usage aggregated from the surviving per-call ledgers (results/*_ledger_*.json): 78.69M input / 12.72M output tokens on deepseek-flash and 1.91M / 0.89M on deepseek-v4-pro, of which extraction is 13.31M / 8.27M over 5,698 calls. That extraction figure undercounts: precompute_sessions.py writes its ledger to a filename keyed on the run scope, so a later re-run overwrote the ledger of the bulk pass, and roughly 15,700 of the 18,464 cached extractions have no surviving per-call record. Extrapolating at the ledgered per-call means puts true extraction nearer 50M input tokens and the flash total nearer 115M. The extraction *cache* is complete regardless—18,464 files, sufficient to rebuild every view offline—it is only the accounting that is partial. End-to-end spend was approximately 340 CNY; that figure is an account-level balance delta, not a per-call sum, since the ledgers record tokens rather than prices.

B Relation rule table

Condition on consecutive chain elements	**Relation**	**Rendering (current / historical)**
attribute in multi-valued vocabulary, or ≥ 3 distinct values in one session	accumulative	all values with dates / same
same session, different value	co-occurring	joined in latest group / joined within step
different session, identical timestamp, different value	contradiction	latest group + [CONFLICT] / step marked /conflict
strictly later timestamp, different value	supersession	latest group only / $v_1 \to v_2 \to v_3$
identical normalized value	merged	—

Table 6: The complete relation rule table. No LLM call is involved in assigning a relation; the table is the implementation.

Multi-valued vocabulary: interest(s), hobby/hobbies, goal(s), plan(s), considering, likes, dislikes, preference(s), activity/activities, emotional state, concern(s), skill(s), wish(es), idea(s), topic(s).

C Query classifier rules

Historical (checked first): `previously`, `before`, `originally`, `at first`, `used to`, `former(ly)`, `old ...(value|name|address|job|phone)`, `first time`, `initial(ly)`, `history of`, `how (has|did) ...change`, `earlier`, `back (then|when)`, `what was ...before`, `previous` not followed by {chat, conversation, discussion, session, message}(s). Current: `current(ly)`, `now`, `latest`, `still`, `today`, `right now`, `as of`, `up-to-date`, `most recent`. Otherwise neutral, routed to the current-state view. Distribution over 500 questions: 44 historical, 31 current, 425 neutral.

References

Zhisheng Chen, Bingfan Zeng, Bangde Cao, Zhengwei Xie, Yuxuan Li, Jinhan Li, and others. MemPrism: Task-Conditioned Relational Memory Views for Long-Horizon Agents. arXiv preprint arXiv:2608.06745, 2026.

Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. arXiv preprint arXiv:2504.19413, 2025.

Xinlong Dai, Jinchuan Zhang, Lei Gao, Xinzhe Hu, Yuefeng He, and Hui Gao. STAIR: Semantic-Temporal Automaton for Interpretable Reasoning in Temporal Question Answering. arXiv preprint arXiv:2608.16224, 2026.

Xinyi Fan, Miri Liu, Ruozhen Yang, Siru Ouyang, and Jiawei Han. Can Agent Memory Systems Track Evolving State? arXiv preprint arXiv:2608.19652, 2026.

Jiazheng Kang, Mingming Ji, Zhe Zhao, and Ting Bai. Memory OS of AI Agent. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 25972–25981, 2025. arXiv:2506.06326.

Yuhang Li and Yuchen Li. What Should an Agent Forget? Separating What Is Stored from What Is Used. arXiv preprint arXiv:2609.10263, 2026.

Ansuman Mullick and Eray Tüzün. Fortunate Recall: Ontology-Driven Memory Lifecycle Management for Persistent Coherence in LLMs. arXiv preprint arXiv:2609.10413, 2026.

Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. Zep: A Temporal Knowledge Graph Architecture for Agent Memory. arXiv preprint arXiv:2501.13956, 2025.

Chen Shen. What Eviction Destroys: A Restore-Counterfactual Audit of Forgetting in Agent Memory. arXiv preprint arXiv:2609.08279, 2026.

Kuan Wang. MemDelta: Controlled Baselines and Hidden Confounds in Agent Memory Evaluation. arXiv preprint arXiv:2606.29914, 2026.

Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2410.10813.

Meriem Yacoubi, Pia Schmidt, Nenad Petrovic, Ahmed Frikha, Martin Kirchhoff, and Alois Knoll. MemoryLACE: Memory Lifecycle-Aware Consolidation and Evidence Retrieval. arXiv preprint arXiv:2609.03201, 2026.

Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. MemoryBank: Enhancing Large Language Models with Long-Term Memory. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38(17), pages 19724–19731, 2024. arXiv:2305.10250.

Yan Zhou, Yue Ouyang, Kaiyang Zheng, and Suncheng Xiang. TEPA: Revoking Stale Memories for Conflict-Robust Language Agents. arXiv preprint arXiv:2608.07429, 2026.